



END OF EXPERIMENT

Twilight Team
Kudaibergen Nazerke
Tolegenova Moldir



GAME CONCEPT

End of Experiment is a 2D pixel-art RPG where the player progresses through a 3-level campaign. The core mechanic involves selecting character-specific ability cards from 4 slots to defeat enemies in turn-based combat.

After 3 attacks, a hero's Ultimate card becomes available. Pills earned from battles are used to upgrade heroes' stats (up to level 3). The player wins when all enemies are defeated, and loses if the party's HP drops to zero. After victory, a reward chest overlay appears. Built for desktop using libGDX.

Structure:

3 levels (max level = 3 in code).

Flow: Hub → Select Level → Combat → Victory/Defeat overlay → Level Transition (walk to portal) → Next Level or Hub.

PATTERNS IN OUR GAME

Singleton (PlayerProgress)

Ensures only one instance of player data (currency, unlocked levels) exists in the entire game. This provides a single source of truth across the hub, combat, and menus, completely preventing bugs where you spend coins on one screen but the balance doesn't update on another.

Factory Method (Hero, Enemy)

Hides the messy logic of object creation. Instead of manually configuring stats every time you need a monster, you simply call `createEnemy("Bat")`, and the factory automatically assigns the correct health, base damage, and attributes for that specific type.

Template Method (Entity)

Creates a rigid skeleton for the attack sequence (prepare → calculate damage → apply post-effects). The overall flow is locked, but child classes like Hero and Enemy can override specific steps, such as adding a random multiplier to their damage calculation.

Builder (Card, LevelData)

Solves the problem of constructors that require too many parameters. It allows you to construct complex objects cleanly step-by-step (e.g., `new Card.Builder().name("Fire").cost(2).damage(50).build()`), making the code highly readable and easy to configure.

Strategy (CombatManager)

Extracts unique combat behaviors (like Sakura's AOE attacks or Gojo's shields) into separate classes. Instead of writing massive if/else blocks inside the Hero class, the game dynamically assigns the correct attack algorithm to the hero at runtime.

PATTERNS IN OUR GAME

Facade (CombatManager)

Acts as a simplified front panel for a complex backend system. The UI (CombatScreen) doesn't need to manually manage turn timers, hand generation, or damage calculations; it just calls a single `playerAttack()` method, and the Facade coordinates all the internal subsystems.

State (TurnManager)

Dictates how the game behaves based on its current phase (e.g., `PLAYER_TURN`, `ENEMY_TURN_ATTACK`). This cleanly controls the combat loop: if the state switches to the enemy's turn, the game automatically ignores player UI clicks and starts the enemy timers.

Command (MovementController)

Wraps input actions into standalone objects (e.g., `MoveUpCommand`). Instead of directly changing X/Y coordinates when 'W' is pressed, the input triggers a command. This makes remapping keys, queueing inputs, or adding new movement mechanics effortless.

Observer / Listener (CombatScreen, etc.)

Decouples different parts of the game using an event-driven approach. When a battle ends, the combat screen doesn't manually load the next level; it simply fires an `onVictory()` event. The main `GameScreen` listens for this event and handles the screen transition independently.



Main Hub

the **Main Hub**. Here, you can manage your three heroes—Sakura, Gojo, and Yuki—track your global currency at the top, and select available levels on the right.



Hero Profile

the Hero Profile. This is a custom book-style overlay where players can review character stats, read ability descriptions, and spend their currency on upgrades.



Combat Screen

Combat Screen. This is a turn-based card system where players drag and drop ability cards onto enemies to attack. Every three attacks charge an ultimate ability. After the player's turn, there is a brief delay before the enemies counterattack with their own unique animations.



Level Transitions

Level Transitions. Instead of a loading screen, players use WASD keys to manually walk their character to the next portal. You can even click the hero avatars at the top to access quick upgrades on the go before stepping into the next fight. Thank you for your attention!"

Lose Overlay



the Lose Overlay
which offers no rewards and returns
you to the hub to try again.

Win Overlay



the Win Overlay,
which rewards you with currency and
lets you proceed to the next stage



THANK YOU FOR ATTENTION!